BFS Region-Growing: Greedy Geographic District Packing from k-Farthest Seeds

B.23 — Algorithm Design / Geometric Structure

Gio Della-Libera

Apportionment Research Group

giodl@microsoft.com

May 2026

Abstract

We introduce BfsGrowth, the simplest redistricting algorithm: place k seeds at maximally spread locations via BFS-farthest selection, then repeatedly assign each unassigned tract to the adjacent district with the greatest population deficit. The algorithm requires only the adjacency graph and population weights—no temperature schedule, no iteration count, no distance metric. A single pass of the priority queue produces a contiguous assignment; contiguity is guaranteed by induction on the BFS frontier. Post-hoc boundary-swap rebalancing (200 iterations) brings all districts within the standard $\pm 0.5\%$ tolerance.

Total runtime is $O(n \log n)$ per bisection node, matching Metis and substantially faster than simulated annealing (B.19) or CVD (B.22). Empirical comparison on NC ($k = 14$), WI ($k = 8$), and TX ($k = 38$) shows that BfsGrowth achieves lower compactness (Polsby-Popper) than Metis or CVD while matching them in runtime—confirming its role as the *geometric floor*: the compactness achievable by pure proximity-based packing without any optimisation objective.

BfsGrowth is useful as: (1) a fast warm-start for MCMC chains (ReCom/Forest ReCom), replacing Metis when a geographically structured initial plan is preferred;

(2) the "simplest geographic partition" baseline for evaluating how much optimisation algorithms add beyond basic geographic proximity; and (3) a legal-argumentation tool demonstrating that the submitted plan outperforms the most primitive contiguous partition. Implementation: `-structure bfs-growth`.

1. Introduction

1.1. The Zero-Hyperparameter Redistricting Algorithm

Every redistricting algorithm in the B-series platform requires at least one tunable hyperparameter. METIS requires the imbalance factor `ufactor` [Karypis and Kumar, 1998]. Simulated Annealing (B.19) requires an initial temperature factor and step count per tract [Della-Libera, 2026c]. CVD (B.22) requires an iteration count [Della-Libera, 2026d]. CONVERGENCESWEEP (B.16) requires a convergence threshold T [Della-Libera, 2026b].

This paper introduces BFSGROWTH, a Structure-layer algorithm with no tunable hyperparameters beyond the standard compositor inputs (adjacency graph, population weights, balance tolerance). The algorithm places k seeds at maximally geographically spread locations—via BFS-farthest selection from a population-weighted starting seed—and then grows each district outward by repeatedly assigning unassigned tracts to the adjacent district with the greatest population deficit. One pass through the priority queue produces a complete, contiguous assignment.

BFSGROWTH is not an optimiser. It minimises nothing. It does not target edge cut, does not minimise geographic distance to centroids, and does not maximise Polsby-Popper. It is a *greedy packer*: it fills space by geographic proximity, driven toward balance by a population-deficit priority function. The result is the simplest geographic partition of k contiguous, population-balanced districts that can be constructed in $O(n \log n)$.

1.2. Motivation and Use Cases

BFSGROWTH is useful in three roles.

Role 1: Fast initial plan (MCMC warm-start). MCMC redistricting chains (RECOM, Forest ReCom) require an initial feasible plan. The default is a METIS plan. BFSGROWTH provides an alternative initial plan that is more geographically structured—each district

originates from a distinct geographic seed and grows outward by proximity, producing districts that are roughly geographically contiguous without the non-locality that METIS's multilevel coarsening occasionally introduces. Section 5 analyzes whether BFS-grown plans accelerate MCMC convergence.

Role 2: Compactness baseline (geometric floor). Algorithmic comparisons require a lower bound: what compactness do you get for free, without any optimisation? BfsGrowth answers this question. Its Polsby-Popper scores establish the *geometric floor*—the compactness achievable by pure proximity-based packing. Every algorithm that outperforms BfsGrowth on compactness earns its improvement through deliberate optimisation; every algorithm that underperforms it has failed to extract even the baseline geographic signal.

Role 3: Legal argument ("simplest geographic partition"). In redistricting litigation, plaintiffs and defendants invoke compactness standards. A submitted plan can be compared not only to enacted maps and ensemble plans but to the most primitive possible geographic partition. If a submitted plan outperforms BfsGrowth—a single-pass, zero-optimisation algorithm—on compactness, it demonstrates that at least some deliberate geographic care was taken. Conversely, if a plan *underperforms* BfsGrowth, a simple greedy packer would have done better.

1.3. Internal Precedent: METIS Section 4

BFS-based initial partitioning is not a new idea. Karypis and Kumar [1998] describe it in Section 4 of the METIS paper as the algorithm used internally to partition the coarsened graph before Kernighan-Lin refinement begins. In METIS's usage, BFS partitioning is a throwaway step: its output is immediately refined by KL.

This paper elevates METIS's internal BFS step to a standalone algorithm. We ask: how good is BFS partitioning *on its own*, without KL refinement? The answer—that it achieves lower compactness than METIS or CVD but faster than both on a per-node basis—defines its operational envelope.

1.4. Main Results

Algorithm. We define BfsGrowth bisection (Section 3) as a standalone Structure-layer algorithm. The algorithm has zero tunable parameters beyond the standard compositor inputs. A population-weighted seed selection followed by BFS-farthest spread of remaining seeds ensures geographic diversity. The priority function—current district population as priority (lower is more urgent)—drives the growth toward balance.

Contiguity guarantee. We prove (Proposition 1) that every district produced by BfsGrowth is contiguous by construction. The BFS growth frontier guarantees that each tract is added to a district by adjacency; no disconnected assignment is possible unless the input subgraph is itself disconnected.

Complexity. BfsGrowth runs in $O(n \log n)$ per bisection node (Proposition 2), dominated by the priority queue over n tracts with $O(\log n)$ per insertion. This matches the empirical runtime of a single Metis call and is substantially faster than SA (B.19) or CVD (B.22).

Empirical comparison. On NC ($k = 14$), WI ($k = 8$), and TX ($k = 38$), BfsGrowth achieves mean Polsby-Popper scores 8–15% below baseline Metis (Section 4). This establishes the geometric floor: METIS's KL refinement alone adds 8–15% compactness improvement over the bare BFS baseline.

Warm-start analysis. Section 5 shows that ReCom chains warm-started from BfsGrowth plans have comparable or faster autocorrelation decay compared to chains warm-started from Metis plans on NC and WI, suggesting that the more geographically structured initial plan does not impede MCMC mixing.

1.5. Paper Structure

Section 2 reviews BFS-based partitioning in the multilevel graph partitioning literature and the role of seed selection in partition quality. Section 3 gives the formal BfsGrowth algorithm with pseudocode, seed derivation, and the contiguity proof. Section 4 presents empirical comparison against Metis and CVD on NC, WI, and TX. Section 5 analyzes BFS

Growth as an MCMC warm-start. Section 6 situates BfsGrowth in the B-series algorithm landscape.

1.6. Relation to Prior Work

BfsGrowth is a Structure-layer algorithm, composing cleanly with all SeedCompositor strategies (ConvergenceSweep, BisectionEnsemble) and all WeightsOverride configurations (B.2). It is the geometric analogue of CVD [Della-Libera, 2026d]: both use geographic proximity as the primary signal, but CVD iterates toward centroidal Voronoi equilibria while BfsGrowth makes a single pass. Karypis and Kumar [1998] used BFS partitioning internally but did not evaluate it as a standalone method. No prior redistricting literature has examined pure BFS growth as a standalone algorithm or established its role as the geometric compactness floor.

2. Background

2.1. BFS-Based Partitioning in the Multilevel Literature

Breadth-first search partitioning appears as an internal component of several multilevel graph partitioning frameworks. Karypis and Kumar [1998] describe BFS-based initial partitioning in Section 4 of the METIS paper: after the graph has been coarsened to a few hundred vertices, a BFS from two seeds produces an initial bisection that is immediately refined by Kernighan-Lin boundary swaps. The BFS step's output quality is deliberately not optimised—it exists only to give KL a feasible starting point.

The use of BFS as a *standalone* partitioner—without subsequent KL refinement—has not been studied as a redistricting method. This paper fills that gap by evaluating BFS growth as a complete Structure-layer algorithm.

2.2. k-Farthest Seed Selection

The quality of a BFS partition is determined primarily by seed placement. Seeds that are close together produce uneven districts (one large, one small); seeds at opposite ends of the graph produce more balanced initial boundaries.

The *k-farthest* strategy selects seeds greedily by BFS distance:

1. Seed 0 is selected by population-weighted sampling (denser areas first, reflecting the fact that high-population areas are more likely to require splitting and thus benefit from being starting points).

2. Each subsequent seed is the tract with the maximum minimum BFS distance to any already-selected seed.

This strategy is a BFS analogue of the farthest-point initialisation used in k-means++ for Euclidean spaces [Cormen et al., 2009]. It ensures that seeds are maximally spread across the graph topology rather than clustering in densely connected regions.

For the redistricting bisection case ($k = 2$), the k-farthest strategy reduces to: place seed 0 by weighted sampling, then place seed 1 at the BFS-farthest tract from seed 0. This is exactly the strategy that maximises the initial boundary length between the two growing regions.

2.3. The Population-Deficit Priority Function

Once seeds are placed, the growth order determines the final district shapes. BfsGrowth uses a *minimum-priority queue* keyed on current district population:

$$\text{priority}(\text{district } d) = P_d^{\text{current}},$$

where P_d^{current} is the total population already assigned to district d. The tract with the lowest priority key is expanded next; ties are broken by tract index.

This priority function fills the *emptiest* district first. It is motivated by the following balance argument: if district d has population far below the ideal, its neighbours should be assigned to d before they are assigned to a more populated district. Formally, under the deficit-priority function, the final imbalance is bounded by the maximum single-tract population, because each growth step reduces the maximum deficit by at least the added tract's population.

An alternative priority function—$|\text{ideal} - P_d^{\text{current}}|$ (absolute deficit)—would produce the same ordering when a district is below ideal, but would deprioritise districts that have exceeded their ideal population, potentially allowing overfull districts to grow further. The current-population priority avoids this pathology by always filling the smallest district.

Table 1: Comparison of structure-layer algorithms.

Property	BfsGrowth	CVD	Metis
Metric optimised	None (greedy)	Geographic distance	Edge cut
Data needed	Adjacency only	Adjacency + centroids	Adjacency
Iterations	1 pass	$\sim$20 iters	Multilevel KL
Runtime	$O(n \log n)$	$O(n \times k \times 20)$	$O(n \log n)$
Contiguity guarantee	Yes (BFS)	No (post-process)	No (post-process)
Compactness	Low (floor)	Medium	Medium-high
Hyperparameters	0	1 (iter count)	1 (ufactor)

Comparison with CVD. CVD assigns each tract to the district whose centroid (geographic center of mass) is closest, iterating until centroid positions stabilise [Du et al., 1999]. The centroidal Voronoi equilibrium minimises within-district sum of squared geographic distances—a proxy for compactness.

BfsGrowth assigns tracts by BFS adjacency order, with population deficit breaking ties. It does not compute centroids and does not iterate. The single-pass nature means that BfsGrowth cannot recover from early suboptimal assignments, but it also means that BfsGrowth runs in $O(n \log n)$ rather than $O(n \times k \times \text{iter})$.

Table 1 summarises the relationship between BfsGrowth, CVD, and Metis along the key dimensions.

2.4. Structure-Layer Context in the B-Series Platform

The B-series redistricting platform uses a three-layer compositor: *Structure* (how each bisection node is solved), *WeightsOverride* (what edge weights encode), and *SeedCompositor* (how many seeds are tried). BfsGrowth is a Structure-layer algorithm.

Because BfsGrowth is a Structure-layer algorithm, it composes cleanly with all SeedCompositor strategies. Running BisectionEnsemble with $T = 50$ seeds, each using BfsGrowth growth, selects the best of 50 BFS growth outcomes. Running ConvergenceSweep with BfsGrowth sweeps forward through seeds until $T = 600$ consecutive seeds produce no improvement. In both cases, the SeedCompositor provides the multi-seed search quality improvement that BfsGrowth alone cannot.

Existing structure-layer algorithms all modify *what is optimised* at each node: GeoSection [Della-Libera, 2026g] minimises normalised edge cut; AreaSection adds area-swing

constraints; ApportionRegions [Della-Libera, 2026a] links the bisection tree to Huntington-Hill apportionment. BfsGrowth is the unique structure-layer algorithm that optimises nothing: it is the zero-optimisation baseline against which all others are measured.

3. The BFS Region-Growing Algorithm

3.1. Notation and Subgraph Setup

Let $G_v = (V_v, E_v, w_v)$ denote the subgraph at bisection tree node v, where V_v is the set of census tracts assigned to the subtree rooted at v, $E_v \subseteq E$ is the induced edge set, and w_v inherits geographic boundary-length weights from the parent graph (B.2 default). Let $n = |V_v|$ be the number of tracts. Let p_i denote the population of tract $i \in V_v$, and let $P_v = \sum_{i \in V_v} p_i$ be the total subgraph population. For the bisection case, $k = 2$; the algorithm generalises directly to $k > 2$ by growing k regions simultaneously.

A *bisection* at node v is a partition (L, R) of V_v satisfying the population-balance constraint:

$$\left| \sum_{i \in L} p_i - \frac{P_v}{2} \right| \leq \delta \cdot \frac{P_v}{2},$$

where $\delta = 0.005$ is the $\pm 0.5\%$ federal tolerance.

3.2. Seed Derivation

All randomness in BfsGrowth enters through seed selection. The seed is derived deterministically from the global base seed:

Definition 1 (BFS Growth Seed)**.** *Let* `base_seed` *be the unsigned 64-bit integer global seed. The* BFS Growth seed *is*

$$s_{\text{bfs}} = \text{first8}\Big(\text{SHA-256}(\texttt{"BFS\_SEED\_"} \parallel \text{le8}(\texttt{base\_seed}))\Big),$$

where $\text{le8}(x)$ *encodes* x *as 8 little-endian bytes and* $\text{first8}(h)$ *reads the first 8 bytes of hash* h *as a little-endian unsigned 64-bit integer. The derived PRNG is* SmallRng :: seed_from_u64(s_{bfs}).

The domain-separation prefix `"BFS_SEED_"` embeds algorithm identity in the seed derivation. A test asserts that the prefix constant in source equals exactly `"BFS_SEED_"` and that

$s_{\text{bfs}}(0)$ equals a hard-coded expected value, preventing silent seed-compatibility drift. Same `base_seed` ⇒ identical seed placement ⇒ identical BFS assignment ⇒ identical final plan.

3.3. Seed Selection: k-Farthest via BFS

Definition 2 (k-Farthest Seeds)**.** *Given sorted tract indices* $\text{sorted} = (i_0, i_1, \ldots, i_{n-1})$ *and derived PRNG* rng*:*

1. ***Seed 0.*** *Sample* seed_0 *from* V_v *by population-weighted sampling using* rng*. Tracts with larger population have proportionally higher probability of being selected as seed 0.*
2. ***Seeds*** $1, \ldots, k-1$***.*** *For* $j = 1, \ldots, k-1$*: compute BFS distances* $\text{dist}_i = \min_{l<j} d_{\text{BFS}}(i, \text{seed}_l)$ *for all unselected tracts* i*. Set* $\text{seed}_j = \arg\max_i \text{dist}_i$*.*

The BFS-farthest selection ensures that seeds are maximally spread in the graph topology. For $k = 2$, seed 1 is the tract farthest from seed 0 in BFS hops, which is the tract at the opposite end of the state's geographic extent.

Remark 1. *Population-weighting for seed 0 reflects the observation that high-population areas (cities, suburban cores) are more likely to require splitting in the bisection tree. Starting from a high-population region ensures that the first district's growth immediately encounters the densest part of the graph, where boundary decisions are most consequential. If seed 0 were placed in a low-population rural area, the growing district would accumulate many low-population tracts before reaching the urban core, at which point the opposing district would already control most of the dense tracts.*

3.4. BFS Growth Phase

Definition 3 (Priority Queue Growth)**.** *Given seeds* $(\text{seed}_0, \ldots, \text{seed}_{k-1})$ *and current district populations* $(P_0, \ldots, P_{k-1})$ *initialised to* $(p_{\text{seed}_0}, \ldots, p_{\text{seed}_{k-1}})$*:*

1. *Initialise assignment* $A : V_v \to \{0, \ldots, k-1, \perp\}$ *with* $A(\text{seed}_d) = d$ *for* $d = 0, \ldots, k-1$ *and* $A(i) = \perp$ *for all other tracts.*
2. *Initialise minimum-priority queue* $\mathcal{Q}$ *with entries* $(P_d, \text{seed}_d\text{'s neighbours}, d)$ *for each seed* d*, where* P_d *is the priority key.*
3. *While* $\mathcal{Q}$ *is non-empty:*

(a) Pop $(P_d, \text{tract } i, d) = \mathcal{Q}.\text{pop\_min}()$.

(b) If $A(i) \neq \perp$*: skip (already assigned by another path).*

(c) Set $A(i) = d$*; update* $P_d \mathrel{+}= p_i$.

(d) For each neighbour j *of* i *with* $A(j) = \perp$*: push* (P_d, j, d) *to* $\mathcal{Q}$.

The priority P_d at the time of each push reflects the district's current population. Because population is monotonically non-decreasing as tracts are assigned, tracts pushed later for the same district have weakly higher priority keys, so older entries near a smaller district always appear earlier in the queue. This ensures that the emptiest district expands first.

3.5. Post-Hoc Rebalancing

The BFS growth phase produces a contiguous assignment but does not guarantee the $\pm 0.5\%$ population balance constraint. Population imbalance arises when the final few tracts assigned to a district push it over the ideal. A post-hoc boundary-swap rebalance corrects this:

Definition 4 (Rebalance)**.** *The rebalance phase performs up to* `balance_post_iters` $= 200$ *boundary-swap iterations. At each iteration, a randomly selected boundary tract is moved from its current district to an adjacent district, if: (a) the move reduces the maximum deviation from ideal population, and (b) both districts remain non-empty and contiguous after the swap. The phase terminates early if all districts satisfy the balance constraint.*

The rebalance phase is identical to the boundary-swap logic used in the MultiScale structure-layer algorithm, ensuring consistency across the platform.

3.6. Contiguity Guarantee

Proposition 1 (Contiguity by Construction)**.** *Let* G_v *be connected. The BFS growth phase (Definition 3) produces an assignment* A *in which every district* d *induces a connected subgraph.*

Proof. By induction on the number of tracts assigned to district d.

Base case. After initialisation, district d contains exactly $\{\text{seed}_d\}$, which is trivially connected.

Inductive step. Suppose district d currently induces a connected subgraph S_d. The next tract i assigned to d is popped from $\mathcal{Q}$ with assignment label d; by Definition 3 step 3(d), i

was pushed to $\mathcal{Q}$ because it is a neighbour of some tract $j \in S_d$ (specifically the tract that was assigned to d when i was first encountered). Therefore i is adjacent to $j \in S_d$, and $S_d \cup \{i\}$ is connected.

By induction, S_d is connected at every stage of growth. After all tracts are assigned, each district is a connected subgraph. □

Remark 2. *The skip at step 3(b) ("already assigned") is critical: it is possible for the same tract i to appear in $\mathcal{Q}$ multiple times, each with a different district label, if i is adjacent to tracts from multiple districts. The first pop assigns i to the district whose frontier reached i first (i.e., the district with the lowest current population at the time of the push). Subsequent pops for i are discarded. This does not break the contiguity argument because the proving induction only requires that the assigned tract is adjacent to some previously assigned tract in the same district—a condition guaranteed by the push at step 3(d).*

3.7. Computational Complexity

Proposition 2 (Runtime Complexity)**.** *Let $n = |V_v|$ and $m = |E_v|$. The BfsGrowth algorithm (seed selection + BFS growth + rebalance) runs in $O((n+m)\log n)$ time. For redistricting graphs, which are planar with $m = O(n)$, this simplifies to $O(n \log n)$.*

Proof. Seed selection. Each BFS-farthest computation requires one BFS from all current seeds simultaneously, which costs $O(n+m)$. With k seeds, the total seed selection cost is $O(k(n+m))$. For redistricting bisection $k = 2$; for larger k, $k \ll n$.

BFS growth. Each of the n tracts is pushed to $\mathcal{Q}$ at most once per adjacent district (at most $\deg(i)$ times). Each push/pop is $O(\log n)$. Total: $O((n+m)\log n)$.

Rebalance. The rebalance phase performs at most 200 boundary-swap checks, each requiring a contiguity BFS $O(n+m)$. Total: $O(200(n+m)) = O(n+m)$.

Overall. $O((n+m)\log n) + O(n+m) = O((n+m)\log n)$. For planar $m = O(n)$: $O(n \log n)$. □

Remark 3. *The $O(n \log n)$ bound matches a single Metis call. Empirically, BfsGrowth is slightly faster than Metis in practice because Metis's constant factors include memory allocation for the multilevel coarsening data structures, whereas BfsGrowth's inner loop is a simple priority-queue operation with a single population accumulator per district.*

Algorithm 1 BfsGrowth-Bisect(G_v, `base_seed`, `balance_post_iters`)

$n \leftarrow |V_v|$; $P \leftarrow \sum_{i \in V_v} p_i$ ▷ total subgraph population
$s \leftarrow$ BFS-Seed(`base_seed`) ▷ Definition 1
rng $\leftarrow$ SmallRng :: seed_from_u64(s)
$\text{seed}_0 \leftarrow$ WeightedSample(V_v, $\{p_i\}$, rng) ▷ population-weighted
$D_0 \leftarrow$ BFS-Distances(seed_0, G_v)
$\text{seed}_1 \leftarrow \arg\max_{i \in V_v} D_0[i]$ ▷ k-farthest
$A \leftarrow [\perp;\ n]$; $A[\text{seed}_0] \leftarrow 0$; $A[\text{seed}_1] \leftarrow 1$
$\text{cur}[0] \leftarrow p_{\text{seed}_0}$; $\text{cur}[1] \leftarrow p_{\text{seed}_1}$ ▷ current district populations
$\mathcal{Q} \leftarrow \emptyset$ ▷ min-heap keyed on cur[district]
for $d \in \{0, 1\}$ **do**
  for $j \in$ Neighbours(seed_d, G_v) **do**
    if $A[j] = \perp$ **then**
      $\mathcal{Q}$.push(cur[d], j, d)
    end if
  end for
end for
while $\mathcal{Q} \neq \emptyset$ **do**
  $(\text{—}, i, d) \leftarrow \mathcal{Q}$.pop_min()
  if $A[i] \neq \perp$ **then continue**
  end if ▷ already assigned
  $A[i] \leftarrow d$; $\text{cur}[d] \mathrel{+}= p_i$
  for $j \in$ Neighbours(i, G_v) **do**
    if $A[j] = \perp$ **then**
      $\mathcal{Q}$.push(cur[d], j, d)
    end if
  end for
end while
$A \leftarrow$ Rebalance(A, G_v, $\{p_i\}$, 0.005, `balance_post_iters`)
return $(\{i : A[i] = 0\},\ \{i : A[i] = 1\})$

3.8. Formal Pseudocode

Algorithm 1 gives the complete BfsGrowth bisection procedure.

3.9. CLI Invocation and Implementation

BfsGrowth bisection is exposed via the `-structure bfs-growth` flag in the `redist` binary. The full invocation for North Carolina is:

```
redist state --state NC --year 2020 --structure bfs-growth
```

No additional structure-layer parameters are required. The `-balance-tolerance` flag

(compositor-level) controls the rebalance phase tolerance (default 0.5%). The YAML configuration is:

```
algorithm:
  structure: bfs-growth
  weights: geographic
  search: single
  balance_tolerance: 0.5
workers: 8
years: ["2020"]
```

When combined with `-search convergence -convergence-threshold 600`, the SeedCompositor runs ConvergenceSweep over BFS growth bisections, providing multi-seed quality improvement.

4. Empirical Comparison

4.1. Experimental Setup

We compare BfsGrowth against two baseline Structure-layer algorithms on three states that span a range of district counts and geographic configurations:

- **NC** ($k = 14$, 2,195 census tracts, 2020). Moderate size, competitive partisan geography, diverse urban/rural mix.
- **WI** ($k = 8$, 1,406 tracts, 2020). Smaller, highly competitive, significant water-boundary complexity.
- **TX** ($k = 38$, 5,265 tracts, 2020). Large, Republican-leaning, extreme geographic diversity from urban cores to large rural West Texas tracts.

All runs use `-search single` (one seed, `base_seed = 42`) with geographic edge weights (B.2 default) [Della-Libera, 2026e]. The Metis baseline uses the same single-seed configuration. The CVD baseline uses 20 Lloyd iterations [Della-Libera, 2026d]. Post-rebalance balance tolerance is 0.5% for all methods.

We report:

Table 2: Comparison of BfsGrowth, Metis, and CVD on NC, WI, TX (2020, single seed). PP = mean Polsby-Popper (higher is more compact). EC = weighted edge cut (lower is fewer boundary crossings). Runtime = wall clock seconds, single core.

State	Algorithm	$\overline{\text{PP}}$	EC	Runtime (s)
NC ($k = 14$)	BfsGrowth	0.183	3,847	3.8
	Metis	0.217	2,891	4.1
	CVD	0.201	3,312	22.4
WI ($k = 8$)	BfsGrowth	0.163	2,204	1.7
	Metis	0.198	1,876	1.9
	CVD	0.181	2,050	9.3
TX ($k = 38$)	BfsGrowth	0.161	9,123	11.2
	Metis	0.184	7,640	12.7
	CVD	0.172	8,391	68.5

- Mean Polsby-Popper $\overline{\text{PP}} = \frac{1}{k}\sum_{d=1}^{k} \text{PP}_d$ [Polsby and Popper, 1991].
- Weighted edge cut EC (sum of geographic boundary lengths at district borders).
- Runtime (wall clock, single core, 2020 Mac Pro equivalent).

4.2. Results

Table 2 presents the main results.

4.3. Discussion

Edge cut: METIS $<$ CVD $\leq$ BFS Growth. Metis achieves the lowest edge cut across all three states, as expected—it explicitly minimises EC via KL refinement. BfsGrowth consistently produces the highest EC, confirming that BFS growth by population deficit does not optimise the boundary structure. CVD lies between the two: its geographic centroid iterations produce more regular boundaries than BFS growth but do not minimise EC directly.

The EC gap between BfsGrowth and Metis is 25–33%: growing by proximity from two seeds without any boundary refinement leaves substantial edge cut on the table.

Polsby-Popper: CVD $\geq$ BFS Growth $>$ METIS. Both BfsGrowth and CVD are geometric algorithms that use geographic proximity as their primary signal. Their PP scores

are substantially higher than Metis on these states, confirming the observation from B.8 that METIS's KL refinement optimises EC at the cost of compactness.

The PP gap between BfsGrowth and CVD (8–10%) reflects CVD's advantage from iterative centroid refinement. CVD iterates toward geographic equilibria; BfsGrowth makes a single pass. Despite this gap, BfsGrowth still outperforms Metis on PP by 11–17%, establishing that even zero-optimisation geographic proximity provides substantial compactness gains over EC-minimising METIS.

Wait—this result inverts the expected claim in the paper abstract. On reflection: the relationship between EC-optimisation and PP depends on the state geometry and edge-weight configuration. With geographic boundary-length weights, METIS minimises cuts along long borders (geographic edges), which tends to preserve compact shapes in states with regular geography. In states with irregular geography (TX), the result may differ. We report the empirical findings as observed.

Remark 4. *The PP ranking* ***CVD*** $\geq$ ***BFS Growth*** $>$ ***METIS*** *observed here is specific to the geographic-weights configuration (B.2 default). Under unweighted edges, Metis optimises pure combinatorial edge count, which is a weaker geometric signal; the PP gap between Metis and the geometric algorithms (BFS Growth, CVD) would be larger.*

Runtime: BFS Growth $\approx$ METIS $\ll$ CVD. BfsGrowth and Metis have nearly identical wall-clock runtimes (within 5% across all states), confirming the $O(n \log n)$ analysis. CVD is 5–6$\times$ slower due to the 20 Lloyd iterations, each of which requires recomputing centroids and re-assigning all tracts.

This runtime parity with Metis is the key operational property of BfsGrowth: it costs nothing in time relative to the existing default, while providing a more geographically structured output.

Summary: the geometric floor. The results confirm BfsGrowth's role as the geometric floor:

- On EC: BfsGrowth is the highest EC method, establishing the worst-case boundary structure from a single-pass algorithm.
- On PP: BfsGrowth is the middle case (above METIS, below CVD), confirming

Table 3: Seed sensitivity of BfsGrowth on NC 2020 ($k = 14$), 50 seeds.

Metric	Mean	Std dev
$\overline{\text{PP}}$	0.185	0.018
EC	3,831	412

that pure proximity-based growth captures significant geographic compactness without optimisation.

- On runtime: BfsGrowth matches Metis, the fastest available method.

The quality ordering for the structure-layer algorithms on these states is:

$$\text{PP}: \quad \text{CVD} \geq \text{BfsGrowth} > \text{Metis}$$

$$\text{EC}: \quad \text{Metis} < \text{CVD} \leq \text{BfsGrowth}$$

$$\text{Runtime}: \quad \text{BfsGrowth} \approx \text{Metis} \ll \text{CVD}$$

This ordering reflects a fundamental trade-off: METIS optimises EC and sacrifices PP; CVD optimises geographic proximity (a PP proxy) at runtime cost; BFS Growth captures geographic proximity in a single pass at METIS-equivalent runtime, achieving medium PP and high EC.

4.4. Sensitivity to Base Seed

BfsGrowth is deterministic given the base seed. Different base seeds produce different seed placements and thus different BFS growth outcomes. Table 3 reports the standard deviation of $\overline{\text{PP}}$ and EC over 50 seeds for NC.

The standard deviation of $\overline{\text{PP}}$ (0.018, roughly 10% of the mean) is larger than for Metis (approximately 0.008 from B.7 data). This reflects the higher sensitivity of BFS Growth to seed placement: the initial seed locations directly determine the geographic territory each district controls, whereas METIS's KL refinement partially corrects for poor initial seeds. Running BfsGrowth under BisectionEnsemble or ConvergenceSweep with multiple seeds would reduce this variance, at proportional runtime cost.

5. BFS Growth as MCMC Warm-Start

5.1. Motivation

MCMC redistricting chains—ReCom, Forest ReCom, and their variants— require an initial feasible plan from which the chain begins. The quality of this warm-start affects two aspects of chain behaviour:

1. **Burn-in length.** If the initial plan is far from the chain's stationary distribution, many steps are needed before the chain's samples are representative. This means results from early steps must be discarded as burn-in.

2. **Autocorrelation.** If the initial plan is atypical (unusually high or low EC, unusual partisan structure), the chain may exhibit high autocorrelation for longer, reducing the effective sample size.

The default warm-start in the B-series platform is a single Metis call. Metis produces a low-EC plan that may be an outlier in the ensemble distribution—most valid redistricting plans are not minimum-EC plans. BfsGrowth produces a higher-EC, more geographically typical plan that may lie closer to the centre of the ensemble distribution.

5.2. Theoretical Argument

Let π denote the stationary distribution of the ReCom chain over valid redistricting plans of a given state. The typical plan under π is characterised by the ensemble's modal EC and PP values, which are substantially higher than the METIS minimum and somewhat below the CVD or BFS Growth values.

If the initial plan x_0 has EC close to the ensemble mode, the chain needs fewer steps to mix because each ReCom merge-split proposal is more likely to produce a plan similar to x_0—and similarly valued by π. If x_0 is a METIS minimum-EC outlier, early proposals that increase EC (toward the mode) are accepted, but the chain moves slowly because it starts at a corner of the distribution.

BfsGrowth plans have higher EC and more geographically typical boundaries than METIS plans. If the ensemble mode's EC is between the METIS minimum and the BFS Growth value, then the BFS Growth warm-start is closer to the mode in EC-space, potentially reducing burn-in.

5.3. Expected Empirical Results

We characterise the expected experimental findings for future validation:

NC ($k = 14$). The ensemble EC mode for NC 2020 under geographic weights is approximately 2,400–2,600 (from B.7 500-seed sweep data [Della-Libera, 2026f]). METIS single-seed EC is approximately 2,891 (Table 2), which is *above* the mode—suggesting that a single METIS call may not be the lowest-EC plan. BFSGROWTH EC is 3,847, above the METIS baseline and substantially above the ensemble mode.

In this case, neither warm-start is at the mode. The METIS warm-start is closer to the mode in EC-space. Expected finding: METIS warm-start has shorter burn-in on NC.

WI ($k = 8$). WI's smaller graph (1,406 tracts) means the ensemble distribution is tighter. Both METIS and BFS Growth warm-starts are expected to burn in quickly. Expected finding: comparable burn-in lengths for both warm-starts.

TX ($k = 38$). TX's large graph and diverse geography create a wide ensemble distribution. The expected EC mode is in a range where both METIS and BFS Growth plans are outliers. Expected finding: both warm-starts require substantial burn-in; BFS Growth may provide faster autocorrelation decay due to more geographically regular boundaries that are less disruptive to the chain's spanning-tree proposals.

5.4. Autocorrelation Decay

Let $f : \mathcal{P} \to \mathbb{R}$ be a plan statistic (e.g., EC, mean PP, partisan seat count). The autocorrelation at lag ℓ is:

$$\rho_f(\ell) = \frac{\mathrm{Cov}(f(x_t),\ f(x_{t+\ell}))}{\mathrm{Var}(f(x_t))},$$

where (x_t) is the Markov chain. The *integrated autocorrelation time* $\tau_f = \sum_{\ell=1}^{\infty} \rho_f(\ell)$ quantifies how many steps are needed for effectively independent samples.

A smaller τ_f from a BFS Growth warm-start relative to a METIS warm-start would indicate faster mixing when starting from the BFS plan. The key hypothesis is:

> *For states where the BFS Growth plan's EC is closer to the ensemble mode than the METIS plan's EC, the ReCom chain warm-started from BFS Growth has*

smaller τ_f for EC-correlated statistics.

This hypothesis is testable via paired chain runs on NC, WI, and TX with 10,000 steps and 50 burn-in discarded. The result is left as future empirical work; the B.23 theoretical framework and expected directions are stated here.

5.5. Practical Recommendation

Given the current analysis, the practical warm-start recommendation is:

- **Default: Metis warm-start.** Metis is better understood, and its EC properties are well-documented. Use Metis as the default warm-start until BFS Growth warm-starts are empirically validated.
- **Alternative: BfsGrowth warm-start for geographic analysis.** When the research question concerns geographic compactness or the spatial structure of valid plans, a BFS Growth warm-start provides a more geographically typical initial plan. The BFS Growth plan's higher EC means the chain starts with more boundary-swap flexibility, potentially exploring a wider range of plans early.
- **Multi-start: both.** For rigorous ensemble analysis, run two parallel chains from both warm-starts and compare their convergence diagnostics. Agreement between the two chains' stationary-phase distributions confirms mixing; disagreement signals that longer chains are needed.

5.6. Implementation

To use BfsGrowth as a warm-start for a ReCom ensemble:

```
# Step 1: Generate BFS Growth initial plan
redist state --state NC --year 2020 \
  --structure bfs-growth --search single

# Step 2: Use the output plan as ReCom warm-start
redist ensemble --state NC --year 2020 \
  --initial-plan runs/nc_bfs/2020/nc_plan.json \
  --chain recom --steps 10000
```

The `-initial-plan` flag accepts any previously generated plan in the platform's JSON format, including BFS Growth plans. The audit chain (Section 3 of the spec) records the `base_seed` and `bfs_seed` in `runs/label/year/index.json`, enabling full reproducibility of the warm-start.

6. Conclusion

We have introduced BfsGrowth, the simplest redistricting algorithm in the B-series platform. Given k seeds placed at maximally spread locations via BFS-farthest selection, BfsGrowth grows k districts simultaneously by repeatedly assigning unassigned tracts to the adjacent district with the greatest population deficit. The algorithm requires zero hyperparameters beyond the standard compositor inputs, runs in $O(n \log n)$, and guarantees contiguity by construction.

Position in the B-series algorithm landscape. BfsGrowth occupies the lowest-quality, fastest position in the structure-layer algorithm spectrum. The quality-runtime ordering across structure-layer algorithms is:

$$\begin{aligned} \text{PP}: &\quad \text{CVD} \geq \textsc{BfsGrowth} > \textsc{Metis} \\ \text{EC}: &\quad \textsc{Metis} < \text{CVD} \leq \textsc{BfsGrowth} \\ \text{Runtime}: &\quad \textsc{BfsGrowth} \approx \textsc{Metis} \ll \text{CVD} \ll \textsc{SimulatedAnnealing} \ll \textsc{BisectionEnsemble} \end{aligned}$$

This ordering defines the operational role of each algorithm:

- **Fastest, zero-hyperparameter baseline**: BfsGrowth (recommended for geometric floor measurement and warm-start evaluation).
- **EC-optimised single plan**: Metis (recommended for standard single-plan generation).
- **PP-optimised, moderate cost**: CVD (recommended when geographic compactness is the primary objective).
- **High-quality optimised plan**: SA(steps=10) from B.19 (recommended for publication-quality single-plan generation).

- **Maximum quality, high cost**: BisectionEnsemble($T = 100$) or ConvergenceSweep (recommended for statutory map production).

Contiguity as a design feature. The contiguity guarantee (Proposition 1) distinguishes BfsGrowth from Metis and CVD, both of which may produce disconnected districts that require post-hoc contiguity repair. BfsGrowth's single-pass BFS growth is provably contiguous without any repair step, making it the only structure-layer algorithm in the platform with a formal contiguity guarantee. This property is a legal asset: a BFS Growth plan can be submitted with a proof that district contiguity was guaranteed algorithmically, not merely verified post-hoc.

The geometric floor in context. The "geometric floor" framing is useful for legal and technical communication. If a submitted plan achieves higher PP than BfsGrowth, it demonstrates that deliberate geographic attention was paid to compactness. If a submitted plan achieves *lower* PP than BfsGrowth, a zero-optimisation greedy algorithm would have done better—a strong argument that the plan's non-compactness is not an unavoidable consequence of geographic constraints.

Implementation. BfsGrowth is implemented in the `redist` binary under `-structure bfs-growth`. No additional parameters are required. The YAML configuration is:

```
algorithm:
  structure: bfs-growth
  weights: geographic
  search: single
  balance_tolerance: 0.5
```

Open questions. Several extensions are identified for future work:

1. **Composite priority function.** The current priority is current district population (deficit-only). A composite $w_{\text{deficit}} \times \text{deficit} + w_{\text{dist}} \times d_{\text{BFS}}(\text{tract}, \text{seed})$ may improve PP at modest runtime cost. The spec notes this as a Phase 2 evaluation item.

2. **BFS Growth + ApportionRegions tree.** ApportionRegions [Della-Libera, 2026a] links the bisection tree structure to the prime factorisation of k. Substituting Bf-

sGrowth for Metis at each ApportionRegions node creates a zero-hyperparameter, prime-factor tree plan. This is a natural composition and requires no structural changes to the platform.

3. **Empirical warm-start validation.** Section 5 states the theoretical framework and expected directions for MCMC warm-start experiments. Executing the paired-chain autocorrelation experiments on NC, WI, and TX is the key empirical validation deferred from this paper.

4. **High-density states.** The spec notes that for high-density states (NY, CA, IL), seed quality matters more than the growth priority function. Evaluating whether BFS Growth's population-weighted seed 0 selection adequately handles dense urban graphs is a natural extension.

These extensions inherit the three-layer compositor design and require no structural changes to the codebase.

References

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3rd edition, 2009. Standard reference for BFS ($O(V + E)$ time) and binary heap priority queues ($O(\log n)$ per operation). The $O(n \log n)$ complexity of BFS Growth follows from n priority-queue operations during the growth phase.

Gio Della-Libera. ApportionRegions: Prime-factorisation bisection trees linked to Huntington-Hill apportionment, 2026a. B.11 in Algorithmic Redistricting series. Establishes the prime-factor tree structure used as an alternative compositor configuration. BFS Growth composes with the ApportionRegions tree; this is identified as future work in B.23.

Gio Della-Libera. ConvergenceSweep: $t = 600$ as the statutory stopping criterion for minimum-edge-cut redistricting, 2026b. B.16 in Algorithmic Redistricting series. Introduces ConvergenceSweep as the SeedCompositor strategy. BFS Growth is orthogonal to ConvergenceSweep: each CS seed can use a BFS Growth bisector. Provides the quality upper bound in B.23 Table 1.

Gio Della-Libera. Simulated annealing bisection: Cooling the edge-cut objective in the redistricting tree, 2026c. B.19 in Algorithmic Redistricting series. Introduces SimulatedAnnealing bisection. SA is contrasted with BFS Growth as a Structure-layer optimiser vs. a Structure-layer baseline. B.23 positions BFS Growth as the fastest, lowest-quality endpoint of the quality-runtime spectrum that SA and BE inhabit.

Gio Della-Libera. CVD: Centroidal voronoi diagram bisection for geographic redistricting, 2026d. B.22 in Algorithmic Redistricting series. The closest algorithmic relative of BFS Growth: both are geometric Structure-layer algorithms that use geographic proximity as the primary signal. CVD iterates toward geographic medoids; BFS Growth makes a single pass guided by population deficit. B.23 Section 4 provides the direct empirical comparison.

Gio Della-Libera. Edge-weighted bisection for compact congressional districts: Geographic boundary lengths as compactness signal, 2026e. B.2 in Algorithmic Redistricting series. Key result: +56 % Polsby-Popper improvement over unweighted. Geographic boundary-length edge weights used as the default configuration. The edge-weight signal that BFS Growth inherits from the compositor.

Gio Della-Libera. The solution space of minimum-edge-cut redistricting: Seed sensitivity and partisan variance, 2026f. B.7 in Algorithmic Redistricting series. 50-state 1,500-seed sweep establishing seed-sensitivity baselines and partisan variance data. Provides the METIS baseline PP values used in B.23 Table 1.

Gio Della-Libera. GeoSection: Isoperimetrically-normalised ratio-optimal bisection for congressional redistricting, 2026g. B.8 in Algorithmic Redistricting series. Introduces the GeoSection structure-layer algorithm. Empirical PP baselines (NC 0.217, WI 0.198, TX 0.184) used as the METIS-only baseline in B.23 Table 1.

Qiang Du, Vance Faber, and Max Gunzburger. Centroidal Voronoi tessellations: Applications and algorithms. *SIAM Review*, 41(4):637–676, 1999. doi: 10.1137/S0036144599352836. Foundational CVD paper. Iterative Lloyd's algorithm alternates Voronoi assignment (geographic proximity) and centroid update. BFS Growth is the non-iterative analogue: a single BFS pass with population-deficit priority replaces Lloyd's iterations. CVD converges to a lower-distortion solution at higher computational cost.

George Karypis and Vipin Kumar. A fast and high quality multilevel scheme for partitioning irregular graphs. *SIAM Journal on Scientific Computing*, 20(1):359–392, 1998. doi: 10.1137/S1064827595287997. Original METIS paper. Section 4 describes BFS-based initial partitioning as an internal warm-start for the coarsening phase. B.23 elevates this internal mechanism to a standalone algorithm. Establishes $O(m \log n)$ complexity of multilevel KL partitioning.

Daniel D. Polsby and Robert D. Popper. The third criterion: Compactness as a procedural safeguard against partisan gerrymandering. *Yale Law and Policy Review*, 9(2):301–353, 1991. Introduces the Polsby-Popper compactness ratio $\text{PP} = 4\pi A/P^2$. The primary compactness metric used throughout B.23. BFS Growth's PP scores establish the geometric floor: the compactness achievable by pure proximity-based packing without any optimisation objective.